Product Requirements Document: Silent Voice

1. Overview

Project Name: Silent Voice
Type: Software Engineering Mini-Project (Agile-based)
Category: Accessibility / Computer Vision / Applied ML

One-line pitch: A real-time, fully offline application that translates static sign language gestures (ASL alphabet + a custom word vocabulary) into text and speech, enabling communication between signers and non-signers.

2. Problem Statement

Deaf and mute individuals frequently face communication barriers in everyday interactions with people who do not know sign language. Existing solutions are often expensive, require internet connectivity, or need specialized hardware. Silent Voice aims to remove these barriers using only a standard webcam and free, open-source tools.

3. Goals

- Recognize 26 ASL alphabet letters (A–Z) in real time via webcam.
- Recognize a custom vocabulary of 15 common word signs.
- Convert recognized signs into readable text, buffered into words/sentences.
- Provide offline text-to-speech output so the translated message can be heard aloud.
- Run entirely on a local machine at zero cost (no paid APIs, no cloud dependency).

4. Non-Goals (Out of Scope)

- Continuous/dynamic motion-based sign language translation (full ASL grammar, facial expressions, two-handed dynamic signs).
- Support for multiple sign language systems (ISL, BSL) in v1.
- Mobile app version (desktop/webcam only for v1).
- Multi-user/cloud sync features.

5. User Roles & User Stories

Signer (Deaf/mute user)

- As a signer, I want my hand gestures translated to text in real time, so that I can communicate with someone who doesn't know sign language.
- As a signer, I want the app to buffer my letters into words, so that I don't have to repeat gestures unnecessarily.
- As a signer, I want a way to add spaces/clear text, so that I can form full sentences.

Listener (non-signer)

- As a listener, I want to see translated text on screen in real time, so that I can follow the conversation.
- As a listener, I want to hear the translated message spoken aloud, so that I don't have to keep watching the screen.

Admin/Trainer

- As an admin, I want to add new gesture samples to the training data, so that the vocabulary can be expanded over time.
- As an admin, I want to retrain the model after adding new data, so that accuracy improves without rebuilding the app.

6. Functional Requirements

ID	Requirement
FR1	System shall capture live webcam video feed
FR2	System shall detect and extract 21 hand landmarks per frame using MediaPipe
FR3	System shall classify landmark data into one of 41 classes (26 letters + 15 words)
FR4	System shall display the predicted letter/word as on-screen text
FR5	System shall buffer stable consecutive predictions into words/sentences
FR6	System shall provide a button/key to trigger text-to-speech playback
FR7	System shall provide a data collection mode for adding new training samples
FR8	System shall allow retraining the model from collected data

7. Non-Functional Requirements

- **Performance:** Prediction latency under 1 second per gesture.
- **Accuracy:** ≥90% validation accuracy on the trained classifier.

- **Cost:** Zero-cost — no paid APIs, no cloud compute, no paid datasets.
- **Portability:** Runs on Windows/Mac/Linux with a standard webcam.
- **Usability:** Simple, uncluttered UI understandable without a manual.

8. Tech Stack

Component	Tool
Language	Python 3.x
Hand Tracking	MediaPipe Hands
Video Capture	OpenCV
Classifier	scikit-learn (Random Forest)
Text-to-Speech	pyttsx3 (offline)
UI	Streamlit
Data Storage	CSV (landmarks), Pickle (trained model)

9. Vocabulary Scope (v1)

Letters: A–Z (26 classes)
Words: hello, thanks, yes, no, help, please, sorry, water, food, stop, more, eat, drink, love, bathroom (15 classes)

10. System Architecture (High Level)

```
Webcam Feed
    │
    ▼
MediaPipe Hand Landmark Extraction (21
points, x/y/z)
    │
    ▼
Feature Vector (63 features)
    │
    ▼
Random Forest Classifier → Predicted Class
    │
    ▼
Prediction Buffer/Stabilizer → Text String
    │
    ├──► Streamlit UI (display)
    └──► pyttsx3 (speech on demand)
```

11. Project Structure

```
silent-voice/
├── collect_data.py       # Webcam capture
+ landmark labeling
```

```
├── train_model.py        # Trains Random
Forest on collected landmarks
├── predict_live.py       # Real-time
inference + text buffering
├── app.py                # Streamlit UI
├── data/
│   └── landmarks.csv
├── model/
│   └── classifier.pkl
├── references/            # Reference
repos (read-only, for pattern inspiration)
├── requirements.txt
└── README.md
```

12. Milestones / Definition of Done

1. **Data Collection** — Script functional; ≥200 samples collected per class.
2. **Training Pipeline** — Model trained; accuracy report generated (≥90% target).
3. **Live Inference** — Real-time prediction working with <1s latency; text buffer functional.
4. **Speech Output** — TTS reads buffered sentence correctly on trigger.
5. **UI** — Streamlit app displays live feed, predicted text, and speak button.
6. **Documentation** — README with setup steps; demo video recorded; user stories & acceptance criteria documented.

13. Acceptance Criteria (Sample)

Feature: Letter Recognition

- Given the signer shows a static ASL letter clearly in frame
- When the hand is held steady for the required number of frames
- Then the system displays the correct letter as text within 1 second

Feature: Text-to-Speech

- Given a sentence has been buffered from recognized signs
- When the listener presses the "Speak" button
- Then the system reads the sentence aloud using offline TTS

14. Future Scope

- Extend to continuous/motion-based signing using LSTM sequence models on landmark streams.
- Add support for ISL and two-handed signs.
- Mobile deployment.
- Auto-suggest/autocomplete for common phrases.